

AFTERNOON EXERCISE — STUDENT INSTRUCTIONS

Mission Control: Variables and Data Types

You're writing telemetry-tracking code for Stellar Outpost, a fictional space station. Mission Control needs a set of small, focused values recorded correctly — nothing here is complicated, but every value needs the right type, and a few need to be updated or converted safely.

Before You Start

- This exercise covers Lesson 4: declaring and assigning variables, updating a variable, all eight primitive data types, and conversion (widening and narrowing).
- You'll be working in `MissionControl.java`. A complete set of tests is already written for you — you never need to write or edit a test yourself.
- Estimated time: 45–75 minutes.

GOOD TO KNOW

Every test will show red/failing the first time you run them — that's expected, not a sign anything is broken. Each TODO you complete correctly should turn its matching test green.

Getting Set Up

1. Open the project your instructor provided in IntelliJ (File → Open, then select the project folder).
2. In the Project panel on the left, expand `src` → `main` → `java` → `org.example` and open `MissionControl.java`.
3. Also open `src` → `test` → `java` → `org.example` → `MissionControlTest.java` so you can see what each test expects — you don't need to edit this file, but reading it is a great way to understand each TODO.
4. If IntelliJ prompts you to “load Maven changes” or shows a small elephant/refresh icon in the top right, click it. This downloads the testing library (JUnit) the project needs.

Your Task

`MissionControl.java` has 11 TODOs, grouped into four sections that match this lesson's objectives:

Section 1 — Declare and Assign (TODO 1)

Write one line of code that declares a variable and assigns it a value, in a single statement.

Section 2 — Update a Variable (TODO 2)

Declare a variable with a starting value, then update it — using a new assignment statement, never a second declaration — to simulate a sensor reading changing over time.

Section 3 — Primitive Data Types (TODOs 3–9)

One TODO for each of boolean, char, byte, short, long, float, and double. Pay close attention to the required suffixes for long and float literals — the exercise is designed to catch this exact mistake if you miss it.

Section 4 — Conversion (TODOs 10–11)

One TODO practicing widening conversion (automatic, safe), and one practicing narrowing conversion (requires an explicit cast, and truncates rather than rounds).

A TIP ON TODO 11

Narrowing a decimal into an int chops off everything after the decimal point. 87.9 becomes 87, not 88 — this trips people up more than any other TODO in this exercise.

Checking Your Work

1. Open MissionControlTest.java.
2. Right-click anywhere in the file and choose Run 'MissionControlTest' (or click the green double-arrow icon next to the class name at the top of the file).
3. A test results panel opens at the bottom of IntelliJ. Green checkmarks mean that test passed; red X's mean it hasn't yet — click a red test to see exactly what value it expected versus what your code returned.
4. You can also run a single test individually by clicking the small green arrow next to that one test method, if you want to check just one TODO at a time.
5. Keep editing MissionControl.java and re-running the tests until everything is green.

Getting Help

- Re-read the TODO comment for that method carefully — it names the exact variable name, type, and value expected.
- Click a failing test to see the “expected vs. actual” values — this almost always tells you exactly what's wrong.
- If you're stuck for more than a few minutes on the same TODO, ask your instructor rather than guessing repeatedly.

Submit your completed project exactly as instructed by your instructor (zipped folder, GitHub link, or LMS upload).